

Assignment 10

Shahriar Hasan

Id: 2010776115

July 17, 2025

1 Generating five images passing five noise vectors/matrices drawn from a normal distribution with mean 5 and variance through the Decoder of your normal Autoencoder.

1.1 Dataset

- Dataset used: MNIST (handwritten digit images)
- Only the training set was used.
- Preprocessing: Pixel values normalized to the range $[0, 1]$, and reshaped to size $(28, 28, 1)$.

1.2 Model Architecture

1.2.1 Encoder

- Input shape: $(28, 28, 1)$
- Layers:
 - Flatten
 - Dense layer with 128 units and ReLU activation
 - Dense layer with 64 units and ReLU activation
- Output: Latent vector of size 64

1.2.2 Decoder

- Input: Vector of shape $(64,)$
- Layers:
 - Dense layer with 128 units and ReLU activation
 - Dense layer with 784 units and Sigmoid activation
 - Reshape to $(28, 28, 1)$
- Output: Reconstructed/generated image

1.2.3 Autoencoder

- Combines encoder and decoder
- Optimizer: Adam
- Loss: Binary crossentropy
- Training: 5 epochs with batch size 256

1.3 Noise Sampling and Image Generation

- Five latent vectors were sampled from a normal distribution $N(\mu = 5, \sigma^2 = 1)$
- Each vector had shape (64,)
- These vectors were passed through the decoder to generate images

1.4 Results

Below are the five generated images:

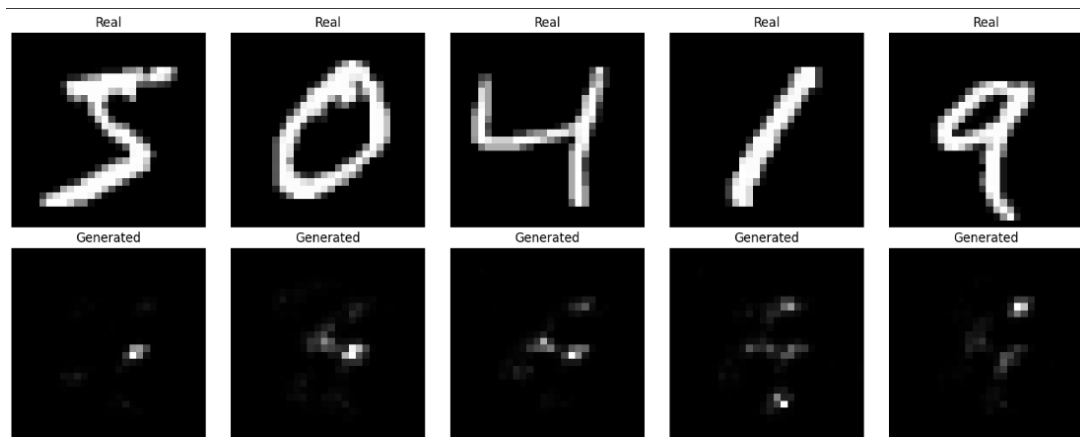


Figure 1: Generated images from latent vectors sampled from $N(5, 1)$

Observations

- The generated images appear abstract or blurry, not resembling clear MNIST digits.
- The decoder was trained on latent vectors centred around 0.
- Sampling from a different latent space (mean = 5) goes outside the learned representation, causing poor generation.

2 Generating five images passing five noise vectors/matrices drawn from a normal distribution with mean 5 and variance through the Decoder of your Denoising Autoencoder.

2.1 Dataset

- Dataset used: MNIST (handwritten digit images)
- Only the training set was used.
- Images were normalized to the range $[0, 1]$.

2.2 Data Preparation

- Gaussian noise was added to the clean training images using a noise factor of 0.5.
- Noisy images were clipped to stay within the range $[0, 1]$.
- These noisy images served as input, and the clean images as output during training.

2.3 Model Architecture

2.3.1 Encoder

- Input shape: $(28, 28, 1)$
- Layers:
 - Flatten
 - Dense layer with 128 units and ReLU activation
 - Dense layer with 64 units and ReLU activation
- Output: Latent vector of size 64

2.3.2 Decoder

- Input: Vector of shape $(64,)$
- Layers:
 - Dense layer with 128 units and ReLU activation
 - Dense layer with 784 units and Sigmoid activation
 - Reshape to $(28, 28, 1)$
- Output: Reconstructed image

2.3.3 Denoising Autoencoder

- Input: Noisy image
- Output: Clean image
- Optimizer: Adam
- Loss: Binary crossentropy
- Trained for 5 epochs with batch size of 256

2.4 Noise Sampling and Generation

- Five latent vectors were sampled from the distribution $\mathcal{N}(\mu = 5, \sigma^2 = 1)$.
- Each vector had shape (64,)
- These vectors were passed through the trained decoder to generate new images.

2.5 Results

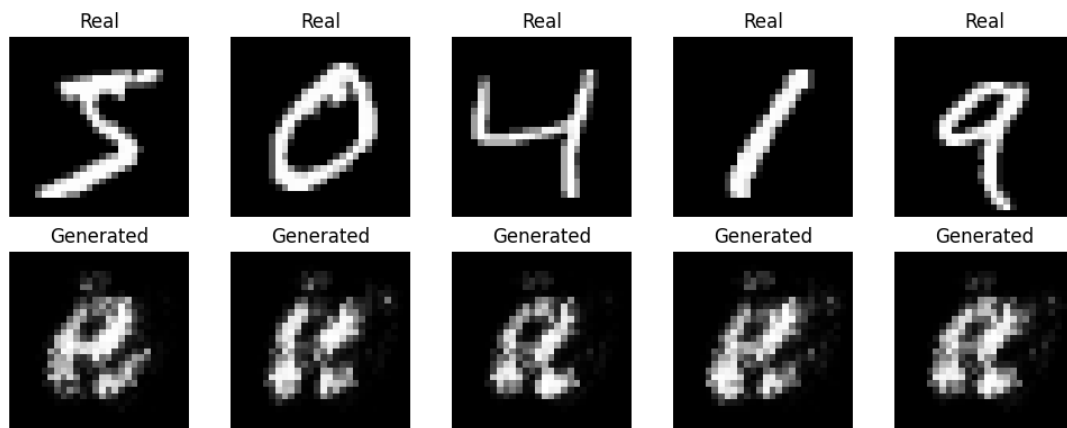


Figure 2: Generated images from latent vectors sampled from $\mathcal{N}(5, 1)$

2.6 Observations

- The generated images appear abstract and do not resemble digits clearly.
- This is expected because the decoder was trained on latent vectors close to 0, while the sampled vectors had mean 5.
- This experiment demonstrates that meaningful generation from a decoder depends on sampling from a latent space distribution similar to the one used during training.

3 Training and evaluating a Variational Autoencoder

3.1 Dataset

We used the **MNIST** dataset, which consists of 70,000 grayscale images of handwritten digits (0–9), each of size 28×28 pixels. The dataset was split into:

- Training set: 60,000 images
- Test set: 10,000 images

The images were normalized to the range $[0, 1]$ and reshaped to $(28, 28, 1)$ to fit the neural network input requirements.

3.2 Variational Autoencoder Architecture

3.2.1 Encoder

The encoder maps the input image to a latent space. Instead of producing a single deterministic latent vector, the VAE outputs two vectors:

- Mean vector (μ or `z_mean`)
- Log-variance vector ($\log \sigma^2$ or `z_log_var`)

These vectors define a **multivariate Gaussian distribution** from which the latent variable z is sampled using the **reparameterization trick**:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

3.2.2 Sampling Layer

A custom Keras layer was used to implement the reparameterization trick, ensuring backpropagation works correctly during training.

3.2.3 Decoder

The decoder reconstructs the original image from the sampled latent vector z . It consists of fully connected layers that reshape the latent representation back into the original image shape ($28 \times 28 \times 1$).

3.3 Loss Function

The total VAE loss combines:

1. **Reconstruction Loss:** Binary cross-entropy between original and reconstructed images, encouraging accurate reconstruction.
2. **KL Divergence:** Kullback-Leibler divergence between the learned latent distribution and a standard normal distribution $\mathcal{N}(0, I)$, acting as a regularizer.

$$\text{Loss} = \text{Reconstruction Loss} + \text{KL Divergence}$$

3.4 Training Configuration

- Epochs: 10
- Batch size: 256
- Optimizer: Adam
- Latent dimension: 2 (for visualization purposes)

The model was trained using a custom training loop by subclassing `tf.keras.Model` and overriding the `train_step()` method.

3.5 Evaluation

3.5.1 Latent Space Visualization

After training, we encoded the test set into the 2D latent space using the encoder. The distribution of the encoded points is visualized using a 2D scatter plot, which shows that different digit clusters occupy distinct regions of the latent space.

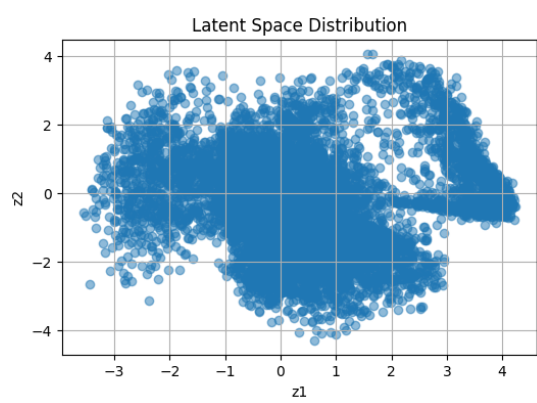


Figure 3: Latent Space Distribution

3.5.2 Image Generation from Latent Space

To evaluate the generative capability of the model, 5 random vectors were sampled from a standard normal distribution $\mathcal{N}(0, 1)$ and passed through the decoder to generate new images. These images resembled handwritten digits, showing that the VAE learned a meaningful latent space.

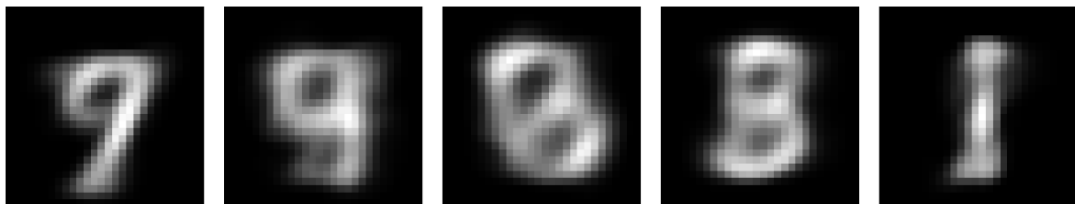


Figure 4: Generated digits from sampled latent vectors

3.5.3 Reconstruction Performance

We also reconstructed test images by passing them through the encoder and decoder. The reconstructed images were visually similar to the originals, confirming the model's ability to learn efficient encodings.

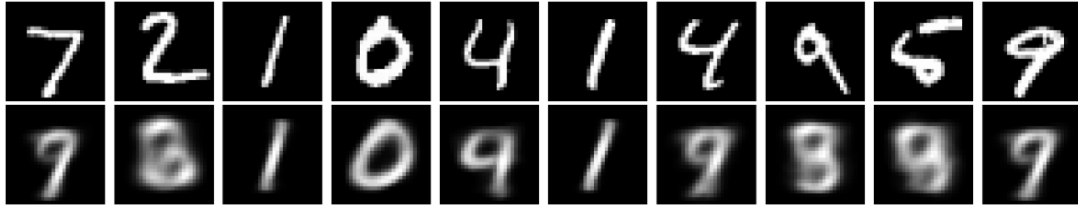


Figure 5: Original (top row) vs. Reconstructed (bottom row) digits

4 Generating five images passing five noise vectors/matrices drawn from a normal distribution with mean 5 and variance through the Decoder of your Variational Autoencoder.

4.1 Methodology

4.1.1 Latent Space Sampling

We defined the latent space with a dimensionality of 2. From this space, we drew five vectors from the normal distribution $\mathcal{N}(5, 1)$. This involves setting:

- Mean $\mu = 5$
- Variance $\sigma^2 = 1$

4.1.2 Image Generation

The five sampled noise vectors were passed through the decoder of a previously trained VAE. The decoder maps each vector from the latent space back into the original image space (28×28 grayscale image).

4.2 Results

The output images generated from the latent vectors drawn from $\mathcal{N}(5, 1)$ are shown below.

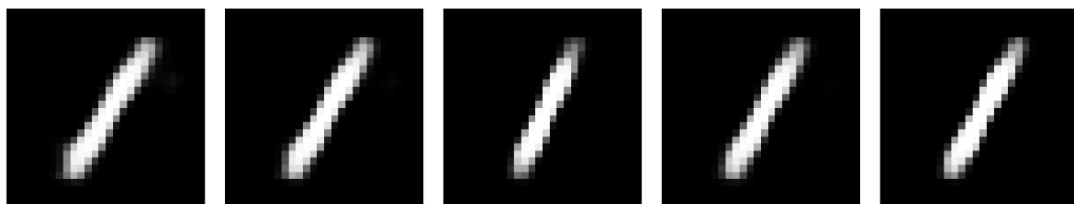


Figure 6: Images generated from latent vectors sampled from $\mathcal{N}(5, 1)$

4.3 Discussion

The decoder of the VAE was trained using latent vectors sampled from a standard normal distribution $\mathcal{N}(0, 1)$. When sampling from $\mathcal{N}(5, 1)$, the latent vectors fall outside the region typically explored during training. As a result, the generated images may appear:

- Less realistic
- Blurred or distorted
- Structurally different from MNIST digits

Despite this, the decoder can still produce images, showing the generalization ability of the latent space.

Code Repository

The complete implementation and source code are available on GitHub: **Github**